

Java Backend Architecture

Interview Series

Chapter 11.10

Database Lock Monitoring

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Chapter 11.10 – Database Lock Monitoring

Introduction

Database lock monitoring is the discipline of observing, diagnosing, and resolving lock contention and deadlocks in production database systems. Undetected lock contention causes cascading slowdowns: a single long-running transaction can block dozens of subsequent writes, causing connection pool exhaustion and application-wide timeouts. PostgreSQL exposes lock state through system views `pg_locks` (current lock grants and waits) and `pg_stat_activity` (session state, current query, wait events). Joining these views reveals the full blocking chain — which session is blocked, which session is the blocker, and what query caused the block. Per-session timeouts (`lock_timeout`, `statement_timeout`) prevent indefinite blocking.

MySQL InnoDB provides equivalent visibility through `performance_schema.data_locks` and `data_lock_waits` (MySQL 8.0+), replacing the deprecated `information_schema` views from MySQL 5.7. The `SHOW ENGINE INNODB STATUS` command reveals the most recent deadlock with full transaction and lock details. Setting `innodb_print_all_deadlocks=ON` logs every deadlock to the MySQL error log for forensic analysis. From the Java side, JDBC lock monitoring involves detecting `LockAcquisitionException` and `SQLTransientException` (wrapped `SQLState 55P03 / 40P01`), implementing retry logic with exponential backoff, and running diagnostic lock queries via a separate monitoring connection to avoid being blocked by the very contention being diagnosed.

PostgreSQL Lock Views – `pg_locks` and `pg_stat_activity`

PostgreSQL Lock Monitoring Views

pg_locks

locktype	relation (OID)	pid
granted	mode	transactionid

JOIN on pid

pg_stat_activity

pid	query	state
wait_event_type	wait_event	xact_start

Lock modes (weakest to strongest):

AccessShareLock	RowShareLock	RowExclusiveLock	ShareUpdateExclusiveLock
ShareLock	ShareRowExclusiveLock	ExclusiveLock	AccessExclusiveLock

Find blocking query:

```
SELECT blocked.pid, blocking.pid AS blocker, blocked.query
FROM pg_stat_activity blocked JOIN pg_stat_activity blocking
ON blocking.pid = ANY(pg_blocking_pids(blocked.pid))
```

Lock Blocking Flow – Detect and Terminate Blocker

Lock Blocking Flow – Identify and Terminate Blocker



Prevent indefinite blocking with per-session timeouts:

SET lock_timeout = '5s'; -- abort if cannot acquire lock within 5s → ERROR 55P03
 SET statement_timeout = '30s'; -- abort statement if running longer than 30s → ERROR 57014
 SET idle_in_transaction_session_timeout = '60s'; -- kill idle transactions

SQLSTATE 55P03 = lock_not_available (lock_timeout exceeded)

SQLSTATE 57014 = query_canceled (statement_timeout exceeded)

pg_terminate_backend() sends SIGTERM; pg_cancel_backend() cancels current query only

MySQL InnoDB Lock Monitoring

MySQL InnoDB Lock Monitoring – performance_schema & INNODB STATUS

performance_schema.data_locks		data_lock_waits (MySQL 8)	
ENGINE_LOCK_ID	ENGINE_TRANSACTION_ID	REQUESTING_ENGINE_LOCK_ID	BLOCKING_ENGINE_LOCK_ID
OBJECT_SCHEMA	OBJECT_NAME	REQUESTING_THREAD_ID	BLOCKING_THREAD_ID
LOCK_TYPE	LOCK_STATUS		

SHOW ENGINE INNODB STATUS -- deadlock section:

LATEST DETECTED DEADLOCK

*** (1) TRANSACTION: TRANSACTION xxx, ACTIVE 0 sec starting index read

*** (1) HOLDS THE LOCK(S): RECORD LOCKS space id X page no Y n bits Z

*** WE ROLL BACK TRANSACTION (2) ← the transaction with fewer locks rolled back

innodb_print_all_deadlocks=ON: logs ALL deadlocks to MySQL error log (not just the latest)

information_schema.INNODB_TRX: shows all currently running InnoDB transactions with trx_started, trx_query, trx_wait_lock_id

LOCK_TYPE: TABLE or RECORD (row-level); LOCK_STATUS: GRANTED or WAITING

MySQL 5.7: used information_schema.INNODB_LOCKS + INNODB_LOCK_WAITS (deprecated in 8.0)

Key Definitions

pg_locks	PostgreSQL system view: one row per lock grant or wait; columns: locktype, relation (OID), pid, granted, mode, transactionid
pg_stat_activity	PostgreSQL view: one row per backend session; columns: pid, query, state, wait_event_type, wait_event, xact_start, query_start
granted (pg_locks)	Boolean: true=lock is held; false=lock is being waited for (session is blocked)
wait_event_type	Category of what the session is waiting for: Lock, LWLock, IO, Client, etc.
pg_blocking_pids(pid)	Returns array of PIDs blocking the given PID; convenient alternative to self-joining pg_locks
pg_terminate_backend(pid)	Sends SIGTERM to the backend; terminates the connection and rolls back its transaction; returns true if signal sent

pg_cancel_backend(pid)	Sends SIGINT to the backend; cancels the current query but keeps the connection open
lock_timeout	Per-session PostgreSQL setting: if a lock cannot be acquired within this duration, raise error 55P03 (lock_not_available)
statement_timeout	Per-session PostgreSQL setting: if a statement runs longer than this, raise error 57014 (query_canceled)
data_locks	MySQL 8.0 performance_schema table: current InnoDB lock grants and waits with ENGINE_LOCK_ID, LOCK_TYPE, LOCK_STATUS, OBJECT_NAME
data_lock_waits	MySQL 8.0 performance_schema table: pairs of (requesting_lock_id, blocking_lock_id); join with data_locks to identify blocker
INNODB STATUS	SHOW ENGINE INNODB STATUS: detailed InnoDB internal state including latest detected deadlock, transaction list, lock waits
innodb_print_all_deadlocks	MySQL global variable: when ON, writes every deadlock (not just the latest) to the MySQL error log for complete audit trail
LockAcquisitionException	Spring/Hibernate exception wrapping SQLSTATE 55P03 (lock_timeout) or 40P01 (deadlock_detected); signals need for retry
INFORMATION_SCHEMA.INNODB_TRX	MySQL table listing all running InnoDB transactions: trx_id, trx_started, trx_state, trx_query, trx_wait_lock_id

PostgreSQL vs MySQL Lock Monitoring Comparison

Feature	PostgreSQL	MySQL InnoDB
Current locks view	pg_locks	performance_schema.data_locks (8.0+)
Lock waits	pg_locks WHERE NOT granted	performance_schema.data_lock_waits
Blocking session	pg_blocking_pids(pid)	JOIN data_lock_waits on blocking_lock_id
Kill connection	pg_terminate_backend(pid)	KILL CONNECTION thread_id
Cancel query only	pg_cancel_backend(pid)	KILL QUERY thread_id
Lock timeout	SET lock_timeout = '5s'	SET innodb_lock_wait_timeout=5
Deadlock log	Logged to pg_log (log_lock_waits)	SHOW ENGINE INNODB STATUS; innodb_print_all_deadlocks
Active transactions	pg_stat_activity WHERE state!='idle'	information_schema.INNODB_TRX

Code Examples

1. PostgreSQL – Find All Blocked Queries and Their Blockers

```
-- Find blocked sessions and their blockers using pg_blocking_pids()
SELECT blocked.pid AS blocked_pid, blocked.username AS blocked_user, blocked.application_name AS blocked_app,
left(blocked.query, 100) AS blocked_query, blocked.wait_event_type AS wait_type,
blocked.wait_event, now() - blocked.xact_start AS blocked_duration, blocking.pid AS blocker_pid,
blocking.username AS blocker_user, left(blocking.query, 100) AS blocker_query,
now() - blocking.xact_start AS blocker_duration FROM pg_stat_activity blocked JOIN
```

```
pg_stat_activity blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid)) WHERE
blocked.wait_event_type = 'Lock' ORDER BY blocked_duration DESC; -- Simpler: just get blocking
PIDs for a specific session SELECT pg_blocking_pids(pg_backend_pid()); -- your own session --
Self-join on pg_locks (explicit, no pg_blocking_pids): SELECT waiting.pid AS waiting_pid,
waiting.query AS waiting_query, blocking.pid AS blocking_pid, blocking.query AS blocking_query
FROM pg_locks w_lock JOIN pg_locks b_lock ON b_lock.granted = true AND w_lock.locktype =
b_lock.locktype AND w_lock.database IS NOT DISTINCT FROM b_lock.database AND w_lock.relation IS
NOT DISTINCT FROM b_lock.relation AND w_lock.transactionid IS NOT DISTINCT FROM
b_lock.transactionid JOIN pg_stat_activity waiting ON waiting.pid = w_lock.pid JOIN
pg_stat_activity blocking ON blocking.pid = b_lock.pid WHERE w_lock.granted = false;
```

2. PostgreSQL – Lock Timeout and Statement Timeout per Session

```
-- Set timeouts for a specific transaction (prevents indefinite blocking) BEGIN; SET LOCAL
lock_timeout = '5s'; -- if can't acquire lock in 5s: ERROR 55P03 SET LOCAL statement_timeout =
'30s'; -- if statement runs >30s: ERROR 57014 -- LOCAL = applies only within this transaction
(reset after COMMIT/ROLLBACK) UPDATE orders SET status='PROCESSING' WHERE id = 123; -- If row
123 is locked by another session for >5s: -- ERROR: canceling statement due to lock timeout
(SQLSTATE 55P03) COMMIT; -- Set as default for a role (persistent): ALTER ROLE app_user SET
lock_timeout = '10s'; ALTER ROLE app_user SET statement_timeout = '60s'; ALTER ROLE app_user
SET idle_in_transaction_session_timeout = '120s'; -- Monitor long-running transactions
(potential blockers): SELECT pid, username, state, wait_event_type, wait_event, now() -
xact_start AS tx_duration, left(query, 120) AS current_query FROM pg_stat_activity WHERE state
!= 'idle' AND xact_start < now() - interval '30 seconds' ORDER BY tx_duration DESC; --
Terminate long-running blocker (requires superuser or pg_signal_backend role): SELECT
pg_terminate_backend(pid) FROM pg_stat_activity WHERE state != 'idle' AND xact_start < now() -
interval '5 minutes' AND pid <> pg_backend_pid(); -- don't kill yourself!
```

3. MySQL 8 – Find Blocking Locks via performance_schema

```
-- MySQL 8: data_locks + data_lock_waits in performance_schema SELECT r.trx_id AS
waiting_trx_id, r.trx_mysql_thread_id AS waiting_thread, r.trx_query AS waiting_query,
b.trx_id AS blocking_trx_id, b.trx_mysql_thread_id AS blocking_thread, b.trx_query AS
blocking_query, dl_wait.OBJECT_NAME AS locked_table, dl_wait.LOCK_TYPE AS lock_type,
dl_wait.LOCK_MODE AS lock_mode, b.trx_started AS blocking_started FROM
information_schema.INNODB_TRX r JOIN performance_schema.data_lock_waits dlw ON
dlw.REQUESTING_ENGINE_TRANSACTION_ID = r.trx_id JOIN performance_schema.data_locks dl_wait ON
dl_wait.ENGINE_LOCK_ID = dlw.REQUESTING_ENGINE_LOCK_ID JOIN information_schema.INNODB_TRX b ON
b.trx_id = dlw.BLOCKING_ENGINE_TRANSACTION_ID; -- Kill the blocking session: KILL CONNECTION
[blocking_thread_id]; -- kills connection and rolls back TX -- or: KILL QUERY [thread_id]; --
cancels current query only, keeps connection -- Show all current InnoDB transactions: SELECT
trx_id, trx_state, trx_started, trx_query, trx_wait_lock_id, trx_lock_structs,
trx_rows_locked, trx_rows_modified FROM information_schema.INNODB_TRX ORDER BY trx_started; --
MySQL lock wait timeout (per session): SET SESSION innodb_lock_wait_timeout = 5; -- wait max 5s
for row lock (default 50)
```

4. MySQL – SHOW ENGINE INNODB STATUS Deadlock Analysis

```
-- Retrieve full InnoDB engine status (includes deadlock info) SHOW ENGINE INNODB STATUS\G; --
Relevant deadlock section output: -- ===== -- LATEST DETECTED DEADLOCK --
===== -- 2025-05-30 10:15:23 0x7f9a3c0 -- *** (1) TRANSACTION: --
TRANSACTION 9821, ACTIVE 0 sec starting index read -- MySQL thread id 42, OS thread handle
140300, query id 1503 localhost app UPDATE -- UPDATE orders SET status='CONFIRMED' WHERE id=100
-- *** (1) HOLDS THE LOCK(S): -- RECORD LOCKS space id 87 page no 4 n bits 72 index PRIMARY of
table `mydb`.`orders` -- trx id 9821 lock_mode X locks rec but not gap -- *** (1) WAITING FOR
```

```
THIS LOCK TO BE GRANTED: -- RECORD LOCKS on table `mydb`.`payments` ... lock_mode X locks rec
but not gap waiting -- *** (2) TRANSACTION: ... UPDATE payments SET ... HOLDS lock on payments
-- *** (2) WAITING FOR THIS LOCK ON orders ... -- *** WE ROLL BACK TRANSACTION (2) -- Enable
logging ALL deadlocks (not just latest): SET GLOBAL innodb_print_all_deadlocks = ON; -- Now
every deadlock appears in /var/log/mysql/mysql_error.log -- Parse deadlock info
programmatically (stored procedure or Java): -- Run SHOW ENGINE INNODB STATUS, extract between
LATEST DETECTED DEADLOCK and next ==== -- Pattern match: "TRANSACTION xxx, ACTIVE", "HOLDS THE
LOCK", "WAITING FOR THIS LOCK"
```

5. Java JDBC – Lock Monitoring and Retry on Lock Timeout

```
@Service public class LockAwareService { @Autowired private DataSource dataSource; // Execute
query with lock timeout and retry on lock contention public <T> T executeWithLockTimeout(String
lockTimeout, int maxRetries, Supplier<T> operation) { int attempt = 0; while (true) { try
(Connection conn = dataSource.getConnection()) { // Set per-session lock timeout before
executing conn.createStatement().execute( "SET LOCAL lock_timeout = '" + lockTimeout + "'");
return operation.get(); } catch (LockAcquisitionException | CannotAcquireLockException ex) { //
Spring wraps SQLSTATE 55P03 (PostgreSQL lock_timeout) // and 40P01 (deadlock_detected) into
these exceptions attempt++; if (attempt >= maxRetries) throw new MaxLockRetriesException("Lock
failed after " + maxRetries, ex); long backoff = 100L * attempt +
ThreadLocalRandom.current().nextLong(50); try { Thread.sleep(backoff); } catch
(InterruptedException ie) { Thread.currentThread().interrupt(); throw new
RuntimeException(ie); } } catch (SQLException ex) { if ("55P03".equals(ex.getSQLState())) { //
lock_not_available – same retry logic attempt++; if (attempt >= maxRetries) throw new
RuntimeException(ex); } else { throw new RuntimeException(ex); } } } } // Run diagnostic lock
query on a separate connection (monitoring) public List<LockInfo> findBlockedSessions() throws
SQLException { String sql = "" SELECT blocked.pid, left(blocked.query, 80) blocked_query,
blocking.pid blocker_pid, left(blocking.query, 80) blocker_query, now() - blocked.xact_start
AS duration FROM pg_stat_activity blocked JOIN pg_stat_activity blocking ON blocking.pid =
ANY(pg_blocking_pids(blocked.pid)) WHERE blocked.wait_event_type = 'Lock' ORDER BY duration
DESC """; try (Connection conn = dataSource.getConnection(); PreparedStatement ps =
conn.prepareStatement(sql); ResultSet rs = ps.executeQuery()) { List<LockInfo> results = new
ArrayList<>(); while (rs.next()) { results.add(new LockInfo(rs.getInt("pid"),
rs.getString("blocked_query"), rs.getInt("blocker_pid"), rs.getString("blocker_query"))); }
return results; } } }
```

6. Spring Boot – Lock Monitoring Actuator and Scheduled Alerting

```
// Custom Actuator endpoint: expose lock info via /actuator/locks @Component @Endpoint(id =
"locks") public class LockMonitoringEndpoint { @Autowired private JdbcTemplate jdbc;
@ReadOperation public Map<String, Object> getLockInfo() { // Blocked sessions List<Map<String,
Object>> blocked = jdbc.queryForList("" SELECT blocked.pid, blocked.username,
left(blocked.query,80) query, now() - blocked.xact_start AS duration, blocking.pid blocker_pid
FROM pg_stat_activity blocked JOIN pg_stat_activity blocking ON blocking.pid =
ANY(pg_blocking_pids(blocked.pid)) WHERE blocked.wait_event_type = 'Lock' ORDER BY duration
DESC LIMIT 20 ""); // Long-running transactions List<Map<String, Object>> longRunning =
jdbc.queryForList("" SELECT pid, username, state, left(query,80) query, now() - xact_start AS
duration FROM pg_stat_activity WHERE xact_start IS NOT NULL AND state != 'idle' AND now() -
xact_start > interval '60 seconds' ORDER BY duration DESC LIMIT 20 ""); return
Map.of("blocked_sessions", blocked, "long_running_transactions", longRunning, "timestamp",
Instant.now().toString()); } } // Scheduled lock alerting @Scheduled(fixedDelay = 30_000)
public void checkForLockContention() { int blockedCount = jdbc.queryForObject( "SELECT count(*)
FROM pg_stat_activity WHERE wait_event_type='Lock'", Integer.class); if (blockedCount > 5) { //
Alert via Slack, PagerDuty, or metrics meterRegistry.gauge("db.blocked.sessions",
blockedCount); alertService.send("HIGH ALERT: " + blockedCount + " sessions blocked on DB
locks"); } }
```

Interview Questions & Answers

Q1. What columns in `pg_locks` are most important for diagnosing lock contention?

The key columns: (1) `pid` — the backend process ID; join with `pg_stat_activity` on `pid` to get the query and user. (2) `granted` — boolean; false means the session is WAITING for the lock (it is being blocked). (3) `mode` — the lock mode requested or held: `AccessShareLock` (SELECT), `RowExclusiveLock` (INSERT/UPDATE/DELETE), `ExclusiveLock` (DROP TABLE), `AccessExclusiveLock` (ALTER TABLE, VACUUM FULL). (4) `locktype` — what is being locked: relation (table/index), tuple (specific row version), transactionid (waiting for a transaction to commit/rollback), advisory (explicit application lock). (5) `relation` — OID of the locked relation; join with `pg_class` to get the table name. (6) `transactionid` — the transaction ID being waited on; when a transaction commits or rolls back, all sessions waiting on its `transactionid` are unblocked. Diagnostic pattern: `WHERE granted = false` shows all currently blocked sessions.

Q2. Explain `pg_stat_activity` and what `wait_event_type='Lock'` means.

`pg_stat_activity` has one row per connected backend (session). Key columns: `pid`, `username` (user), `application_name`, `state` (active/idle/idle in transaction/idle in transaction (aborted)), `query` (current or last query), `xact_start` (when the current transaction started), `query_start` (when the current query started), `wait_event_type` (category of wait), `wait_event` (specific wait). `wait_event_type='Lock'` means the session is blocked waiting to acquire a lock. `wait_event` in this case is the lock mode being waited for (e.g., 'relation', 'tuple', 'transactionid'). Other important `wait_event_types`: 'LWLock' (internal lightweight lock, not application-level), 'IO' (waiting for disk I/O), 'Client' (waiting for client to send query). Sessions with `state='idle in transaction'` are holding transactions open without executing queries — dangerous because they hold locks and prevent VACUUM.

Q3. How do you find which session is blocking another in PostgreSQL?

Three approaches: (1) `pg_blocking_pids(pid)` — simplest: `SELECT pg_blocking_pids(42)` returns an array of PIDs blocking session 42. (2) Self-join on `pg_stat_activity` using `pg_blocking_pids`: `SELECT blocked.pid, blocking.pid FROM pg_stat_activity blocked JOIN pg_stat_activity blocking ON blocking.pid = ANY(pg_blocking_pids(blocked.pid)) WHERE blocked.wait_event_type = 'Lock'`. (3) Self-join on `pg_locks` directly: join `pg_locks l1` (WHERE NOT l1.granted) with `pg_locks l2` (WHERE l2.granted) on matching `locktype`, `relation`, `transactionid`. This reveals the exact lock mode conflict. Best practice: always join with `pg_stat_activity` to see the current query of both the blocked session (to understand what it needs) and the blocker (to understand what it is doing and how long it has been running). Include `xact_start` to know how long the blocker has held its lock.

Q4. What is `lock_timeout` and how does it prevent connection pool exhaustion?

`lock_timeout` is a per-session PostgreSQL setting that aborts a statement if it cannot acquire a required lock within the specified duration (e.g., `SET lock_timeout = '5s'`). When exceeded: `SQLSTATE 55P03` (`lock_not_available`), error message 'canceling statement due to lock timeout'. Without `lock_timeout`: a slow UPDATE that waits for a lock held by a long transaction waits indefinitely. Meanwhile, the application connection pool thread is occupied. More requests arrive, each gets a connection, all queue behind the same lock — connection pool is exhausted. Application returns 503/500 to clients. With `lock_timeout=5s`: after 5s, the blocked query fails with 55P03. The application catches it and retries (or returns an error quickly). The connection is returned to the pool. Connection pool stays healthy. Set `lock_timeout` per-transaction with `SET LOCAL` (not global) so it only affects the current operation.

Q5. What is the difference between `pg_terminate_backend` and `pg_cancel_backend`?

`pg_terminate_backend(pid)`: sends SIGTERM to the backend process. This terminates the connection entirely, rolls back any in-progress transaction, releases all locks, and closes the connection. The client receives an 'ERROR: terminating connection due to administrator command'. The backend exits. Use when you need to forcibly end a session (e.g., long-running idle transaction blocking VACUUM or another transaction). `pg_cancel_backend(pid)`: sends SIGINT to the backend. This cancels the CURRENT QUERY but keeps the connection and transaction open. The client receives 'ERROR: canceling statement due to user request'. The backend remains connected. Use when you want to stop a runaway query without losing the session. Both require the caller to be a superuser or have the `pg_signal_backend` role. Neither works on superuser backends from non-superusers. Verify with `SELECT pg_terminate_backend(pid)` — returns true if signal sent, false if session already gone.

Q6. Explain the MySQL performance_schema.data_locks and data_lock_waits tables.

`data_locks` (MySQL 8.0+): one row per lock held or waited for. Key columns: `ENGINE_LOCK_ID` (unique lock identifier), `ENGINE_TRANSACTION_ID` (transaction holding or requesting the lock), `OBJECT_SCHEMA`, `OBJECT_NAME` (table), `INDEX_NAME`, `LOCK_TYPE` (TABLE or RECORD — row-level), `LOCK_MODE` (S=shared, X=exclusive, IS=intention-shared, IX=intention-exclusive; GAP=gap lock, REC_NOT_GAP=record-only), `LOCK_STATUS` (GRANTED or WAITING). `data_lock_waits`: one row per blocked-by-blocker pair. Columns: `REQUESTING_ENGINE_LOCK_ID` (the blocked lock), `BLOCKING_ENGINE_LOCK_ID` (the lock causing the block), `REQUESTING_THREAD_ID`, `BLOCKING_THREAD_ID`. Join these tables with `information_schema.INNODB_TRX` to get the full picture: which transaction is blocked, which is blocking, what query each is running. This replaced the deprecated `INFORMATION_SCHEMA.INNODB_LOCKS` and `INNODB_LOCK_WAITS` from MySQL 5.7.

Q7. How do you read the SHOW ENGINE INNODB STATUS deadlock output?

The LATEST DETECTED DEADLOCK section shows: (1) Timestamp of the deadlock. (2) TRANSACTION (1): transaction ID, state, thread ID, and current query. (3) HOLDS THE LOCK(S): the lock that Transaction 1 currently holds (shown as RECORD LOCKS space id N page no M — identifies the specific InnoDB page and record). (4) WAITING FOR THIS LOCK: the lock Transaction 1 is waiting for. (5) TRANSACTION (2): same structure for the other transaction in the deadlock. (6) WE ROLL BACK TRANSACTION (N): InnoDB chose to roll back one transaction (typically the one with fewer row locks or shorter duration) to break the deadlock. Key analysis: identify what tables each transaction locked (HOLDS) and what they need (WAITING FOR). The lock cross-dependency creates the cycle. Fix: reorder lock acquisition order (always lock tables in the same order), or break into smaller transactions, or use `SELECT FOR UPDATE SKIP LOCKED`.

Q8. What does innodb_print_all_deadlocks=ON do and when should you enable it?

By default, `SHOW ENGINE INNODB STATUS` only shows the MOST RECENT deadlock. If your application experiences multiple deadlocks per minute, you lose history — each new deadlock overwrites the previous. `innodb_print_all_deadlocks=ON`: logs every detected deadlock in full detail to the MySQL error log (`/var/log/mysql/error.log` or configured `log_error` path). Log format is identical to the `SHOW ENGINE INNODB STATUS` output. Enable in production when: (1) You are actively investigating deadlock patterns. (2) Deadlocks are intermittent and hard to catch with manual monitoring. (3) You need a complete audit trail for compliance or debugging. Caution: if deadlock frequency is very high, log file can grow rapidly. Monitor disk usage. Disable once the issue is diagnosed: `SET GLOBAL innodb_print_all_deadlocks = OFF`. Persist in `my.cnf` for server restart.

Q9. How do you implement lock monitoring in a Java Spring Boot application?

Implementation approaches: (1) Actuator endpoint: create a `@Endpoint(id='locks')` class with a `@ReadOperation` that queries `pg_locks` and `pg_stat_activity` via `JdbcTemplate`. Expose at `/actuator/locks` (secure with `management.endpoints.web.exposure`). (2) Scheduled alerting: `@Scheduled(fixedDelay=30000)` method that queries blocked session count; if above threshold, emit a Micrometer gauge and trigger alert. (3)

Exception-based monitoring: `@ExceptionHandler` for `LockAcquisitionException` (Spring's wrapper for `SQLSTATE 55P03` and `40P01`); increment a Counter in the handler to track lock contention frequency per operation. (4) Connection pool metrics: HikariCP exposes `hikari.connections.pending` (connections waiting for a pool slot) — a spike here often accompanies lock contention. (5) Dedicated monitoring connection: create a separate `DataSource` for monitoring queries (not in the main pool) so monitoring queries can run even when the main pool is exhausted due to lock contention.

Q10. What SQLSTATE codes correspond to lock-related errors in PostgreSQL?

Key PostgreSQL `SQLSTATE` codes for locking: (1) `55P03` — `lock_not_available`: raised when `lock_timeout` is exceeded and the lock cannot be acquired in time. Message: 'canceling statement due to lock timeout'. (2) `40P01` — `deadlock_detected`: raised when PostgreSQL's deadlock detector identifies a cycle of blocked transactions and chooses one to abort. Message: 'deadlock detected'. (3) `57014` — `query_canceled`: raised when `statement_timeout` is exceeded, or when `pg_cancel_backend()` is called. Message: 'canceling statement due to statement timeout'. (4) `25P02` — `in_failed_sql_transaction`: if a transaction had an error, all subsequent statements fail with this code until `ROLLBACK`. In Spring: `LockAcquisitionException` wraps `55P03` and `40P01`; `CannotSerializeTransactionException` wraps `40001` (`serialization_failure`). All are subtypes of `TransientDataAccessException` — indicating a retry is appropriate.

Q11. How do advisory locks work in PostgreSQL and when would you use them?

Advisory locks are application-level locks explicitly requested by the application (not automatically acquired by SQL statements). They are identified by a 64-bit integer or a pair of 32-bit integers. Functions: `pg_try_advisory_lock(key)` — returns true if lock acquired, false if already taken (non-blocking). `pg_advisory_lock(key)` — blocks until acquired. `pg_advisory_unlock(key)` — releases. `pg_advisory_lock_shared(key)` — shared mode. Advisory locks are visible in `pg_locks` with `locktype='advisory'`. Use cases: (1) Distributed leader election — one instance acquires advisory lock; if it dies, lock is released and another instance wins. (2) Preventing concurrent execution of a scheduled job across multiple app instances. (3) Application-level resource locking that doesn't map to a specific table row. Advantages over row locks: work across tables, don't require a specific row to exist, don't block DDL. Use `pg_try_advisory_lock` in Spring Scheduled methods to ensure single-instance execution.

Q12. What causes lock contention in high-concurrency applications and how do you prevent it?

Common causes: (1) Long transactions holding row locks while doing unrelated work (calling external APIs, waiting for user input). (2) Table-level locks from DDL (`ALTER TABLE`, `CREATE INDEX` without `CONCURRENTLY`) blocking all DML. (3) Hot rows — frequently updated rows (counters, status fields) that every request needs to lock. (4) Missing indexes — `UPDATE/DELETE` without an index does a full sequential scan, locking many rows. (5) Cascading FK updates — `UPDATE` on a parent table locks child rows via FK constraints. Prevention: (1) Keep transactions short — no external calls inside transactions. (2) Use `CREATE INDEX CONCURRENTLY`. (3) Replace hot row updates with append patterns (counter CRDT, event sourcing). (4) Add indexes on all filtered columns in `UPDATE/DELETE WHERE` clauses. (5) Set `lock_timeout` to bound maximum wait time. (6) Use `SELECT ... FOR UPDATE SKIP LOCKED` for queue processing — skip locked rows rather than blocking.

Q13. Explain SELECT FOR UPDATE SKIP LOCKED and its use in queue processing.

`SELECT FOR UPDATE SKIP LOCKED` is a PostgreSQL (and MySQL InnoDB) feature that selects rows and locks them, but SKIPS rows that are already locked by another transaction (instead of blocking). This is ideal for work-queue processing: multiple workers each `SELECT` a batch of unprocessed jobs `FOR UPDATE SKIP LOCKED`. Worker A gets rows 1-10 (locked), Worker B issues the same query — skips rows 1-10 (locked), gets rows 11-20. No blocking, no deadlocks, high throughput. Usage: `SELECT * FROM jobs WHERE status='PENDING' ORDER BY created_at LIMIT 10 FOR UPDATE SKIP LOCKED`. After processing: `UPDATE jobs SET status='DONE' WHERE id IN (...)`. Compared to `NOWAIT`: `SKIP LOCKED` skips locked

rows (returns only available rows); NOWAIT returns an error immediately if ANY requested rows are locked. SKIP LOCKED enables efficient parallel processing without contention.

Q14. How do you diagnose connection pool exhaustion related to lock contention?

Symptoms: application returns 503 errors; HikariCP 'Connection is not available, request timed out after 30000ms'. Diagnosis steps: (1) Check blocked sessions: `SELECT count(*) FROM pg_stat_activity WHERE wait_event_type='Lock'`. A high count indicates lock contention blocking connections. (2) Check pool pending: HikariCP metric `hikari.connections.pending > 0` means connections waiting to be acquired from the pool. (3) Find the root blocker: the session holding the lock that all others are waiting for (`pg_blocking_pids` chain leads to a session with no blockers itself). (4) Check blocker's query duration: how long has it been running? Is it stuck? (5) Check if it is idle in transaction: `state='idle in transaction'` means the transaction started but no query is executing — blocked on app-side code. Resolution: set `idle_in_transaction_session_timeout` to kill these. Long-term fix: keep transactions short, set `lock_timeout`, size the connection pool appropriately.

Q15. What is a deadlock and how does PostgreSQL detect and resolve it?

A deadlock occurs when two or more transactions form a circular dependency of lock waits: TX_A holds lock on Row 1, waits for Row 2. TX_B holds lock on Row 2, waits for Row 1. Neither can proceed. PostgreSQL has a deadlock detector that runs periodically (controlled by `deadlock_timeout`, default 1 second). When a session waits for a lock longer than `deadlock_timeout`, PostgreSQL checks for cycles in the wait-for graph. If a cycle is detected, it chooses one transaction to abort (typically the one that is cheapest to abort), raises `SQLSTATE 40P01 (deadlock_detected)` for that transaction, which causes it to roll back all changes and release its locks. The other transaction(s) can then proceed. Application response: catch `DeadlockLoserDataAccessException` (Spring) or check `SQLState 40P01`; rollback completely and retry with exponential backoff. Prevention: always acquire locks in the same order across transactions (e.g., always lock account A before account B).

Q16. How do you monitor lock wait times and set up alerting?

Metrics to collect: (1) Number of blocked sessions: `SELECT count(*) FROM pg_stat_activity WHERE wait_event_type='Lock'`. (2) Maximum lock wait duration: `SELECT max(now() - query_start) FROM pg_stat_activity WHERE wait_event_type='Lock'`. (3) Deadlock count: set `log_lock_waits=on` in `postgresql.conf` + `log_min_duration_statement=0`; parse the PostgreSQL log for 'deadlock detected' entries. (4) Lock timeout count: parse log for 'canceling statement due to lock timeout'. (5) HikariCP metrics: `hikari.connections.pending` via Micrometer. Alerting thresholds: (a) `blocked_sessions > 5` for 60 seconds → warning. (b) `max_lock_wait > 10s` → critical. (c) `deadlock_rate > 1/minute` → investigate. (d) `hikari.connections.pending > 0` for 30s → alert. Implementation: Micrometer gauge updated by a `@Scheduled` method querying `pg_stat_activity`; export to Prometheus → Grafana dashboard + PagerDuty alert rules.

Q17. What is the InnoDB lock wait timeout and how does it differ from PostgreSQL's lock_timeout?

InnoDB lock wait timeout (`innodb_lock_wait_timeout`): controls how long a transaction waits for a row-level lock before giving up. Default: 50 seconds. When exceeded: `ERROR 1205 (HY000): Lock wait timeout exceeded; try restarting transaction`. The transaction is NOT fully rolled back — only the current statement fails (by default). Set `innodb_rollback_on_timeout=ON` to rollback the entire transaction on timeout. Set per session: `SET SESSION innodb_lock_wait_timeout = 5`. PostgreSQL `lock_timeout`: applies to ALL lock types (row, table, advisory, etc.), not just row locks. Default: 0 (disabled). Raises `SQLSTATE 55P03` immediately when timeout exceeded. The statement is cancelled; the transaction is NOT rolled back (can continue or rollback manually). Key difference: InnoDB default is 50s (very long for production); PostgreSQL default is no timeout (infinite wait). In production: set both explicitly per-session — PostgreSQL: `SET LOCAL lock_timeout='5s'`; MySQL: `SET SESSION innodb_lock_wait_timeout=5`.

Q18. How do you prevent accidental table-level locks during schema migrations?

Schema migration risks: ALTER TABLE in PostgreSQL acquires AccessExclusiveLock — blocks ALL reads and writes for the duration of the migration. Strategies: (1) CREATE INDEX CONCURRENTLY — builds an index without locking the table. Takes longer but does not block production traffic. (2) ADD COLUMN with a default value: in PostgreSQL 11+, ADD COLUMN with a non-volatile default no longer rewrites the table (stored as a table default, not in each row). Earlier versions: split into ADD COLUMN nullable, backfill in batches, ADD NOT NULL constraint. (3) Set lock_timeout before ALTER: if the migration cannot acquire the lock within N seconds, it fails fast rather than blocking all traffic for minutes. (4) Use pg_repack to perform space reclamation and table rewrites online (pg_repack acquires minimal locks). (5) Perform migrations during low-traffic windows with pre-applied lock_timeout and monitoring. (6) Liquibase / Flyway: add SET LOCAL lock_timeout in migration scripts. (7) Shadow table pattern for major rewrites.

Q19. What information does information_schema.INNODB_TRX provide and how do you use it?

INNODB_TRX lists all currently running InnoDB transactions. Key columns: trx_id (unique transaction identifier), trx_state (RUNNING, LOCK WAIT, ROLLING BACK, COMMITTING), trx_started (when the transaction began), trx_requested_lock_id (the lock it is waiting for, if blocked), trx_wait_started (when it started waiting), trx_weight (number of lock structs + rows modified — used by deadlock resolver to choose victim), trx_query (current SQL statement being executed), trx_rows_locked (rows locked), trx_rows_modified (rows modified in this transaction). Use cases: (1) Find long-running transactions: WHERE trx_started < now() - INTERVAL 60 SECOND. (2) Find transactions in LOCK WAIT: WHERE trx_state='LOCK WAIT'. (3) Identify high-impact transactions (large trx_weight) for targeted kill. (4) Join with data_lock_waits and data_locks for the full blocking picture. (5) Monitor via JDBC in Spring: scheduled query every 30s, alert if trx_rows_locked > threshold or wait duration > threshold.

Q20. How would you design a comprehensive database health dashboard for lock monitoring?

Dashboard components for PostgreSQL: (1) Blocked sessions panel: real-time count of pg_stat_activity WHERE wait_event_type='Lock'; time series over last 1 hour. (2) Lock wait heatmap: blocked sessions by table name (join pg_locks with pg_class); identify hot tables. (3) Long-running transactions: sessions with xact_start older than 60s; show user, query, duration. (4) Idle-in-transaction: sessions in 'idle in transaction' state > 30s — dangerous for VACUUM and lock accumulation. (5) Deadlock rate: parse PostgreSQL logs or count LockAcquisitionException metrics. (6) Connection pool usage: HikariCP active/idle/pending gauges. MySQL additions: (7) InnoDB lock wait graph: transactions in LOCK WAIT state + blocking transaction; duration. (8) Deadlock log tail: last 10 entries from innodb_print_all_deadlocks log file. Implementation: Grafana + Prometheus; custom Prometheus exporter running lock queries every 15s; Spring Boot metrics via Micrometer; PagerDuty integration for threshold alerts on blocked_sessions > 10 or lock_wait_duration > 30s.